

Лекция. Храните и получайте доступ к данным с помощью ключей с помощью DataStore

Сохраняйте предпочтения локально с помощью DataStore

В этом разделе вы узнали, как использовать SQL и Room для локального хранения данных на устройстве. SQL и Room - это мощные инструменты. Однако в тех случаях, когда вам не нужно хранить реляционные данные, DataStore может стать простым решением. Компонент DataStore Jetpack - это отличный способ хранить небольшие и простые наборы данных с минимальными накладными расходами. DataStore имеет две различные реализации: Preferences DataStore и Proto DataStore.

Preferences DataStore хранит пары ключ-значение. В качестве значений могут выступать основные типы данных Kotlin, такие как String, Boolean и Integer. В нем не хранятся сложные наборы данных. Он не требует предопределенной схемы. Основное применение Preferences DataStore - хранение предпочтений пользователя на его устройстве. Proto DataStore хранит пользовательские типы данных. Для него требуется предопределенная схема, которая сопоставляет определения типов данных с объектными структурами. В этом коделабе рассматривается только Preferences DataStore, но вы можете прочитать больше о Proto DataStore в документации по DataStore.

Preferences DataStore - это отличный способ хранения настроек, контролируемых пользователем, и в этом мастер-классе вы узнаете, как реализовать DataStore для этого!

Что надо знать

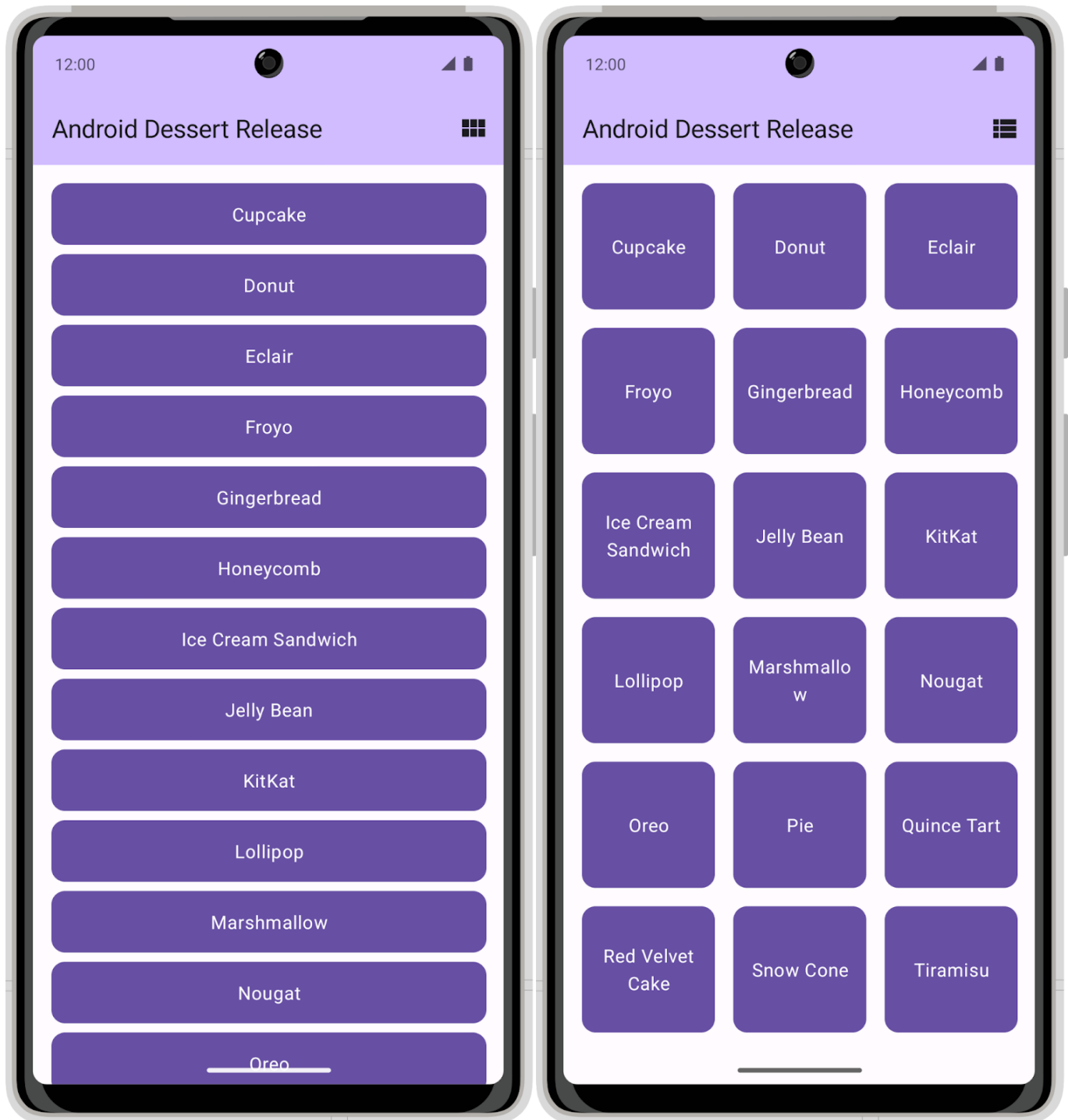
- Завершите курс «Основы Android с Compose», пройдя кодельную лабораторию «Чтение и обновление данных с Room».

Что вам понадобится

- Компьютер с доступом в интернет и Android Studio
- Устройство или эмулятор
- Стартовый код для приложения Dessert Release

Что вы будете создавать

- Приложение **Dessert Release** отображает список релизов Android. Иконка на панели приложения переключает макет между представлением сетки и представлением списка.



В текущем состоянии приложение не сохраняет выбор макета. Когда вы закрываете приложение, выбор макета не сохраняется, и настройка возвращается к выбору по умолчанию. В этом кодовом примере вы добавляете DataStore в приложение **Dessert Release** и используете его для сохранения предпочтений выбора макета.

Загрузите стартовый код

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-dessert-release.git
$ cd basic-android-kotlin-compose-training-dessert-release
$ git checkout starter
В Android Studio откройте папку basic-android-kotlin-compose-training-dessert-
```

release.

Откройте код приложения Dessert Release в Android Studio.

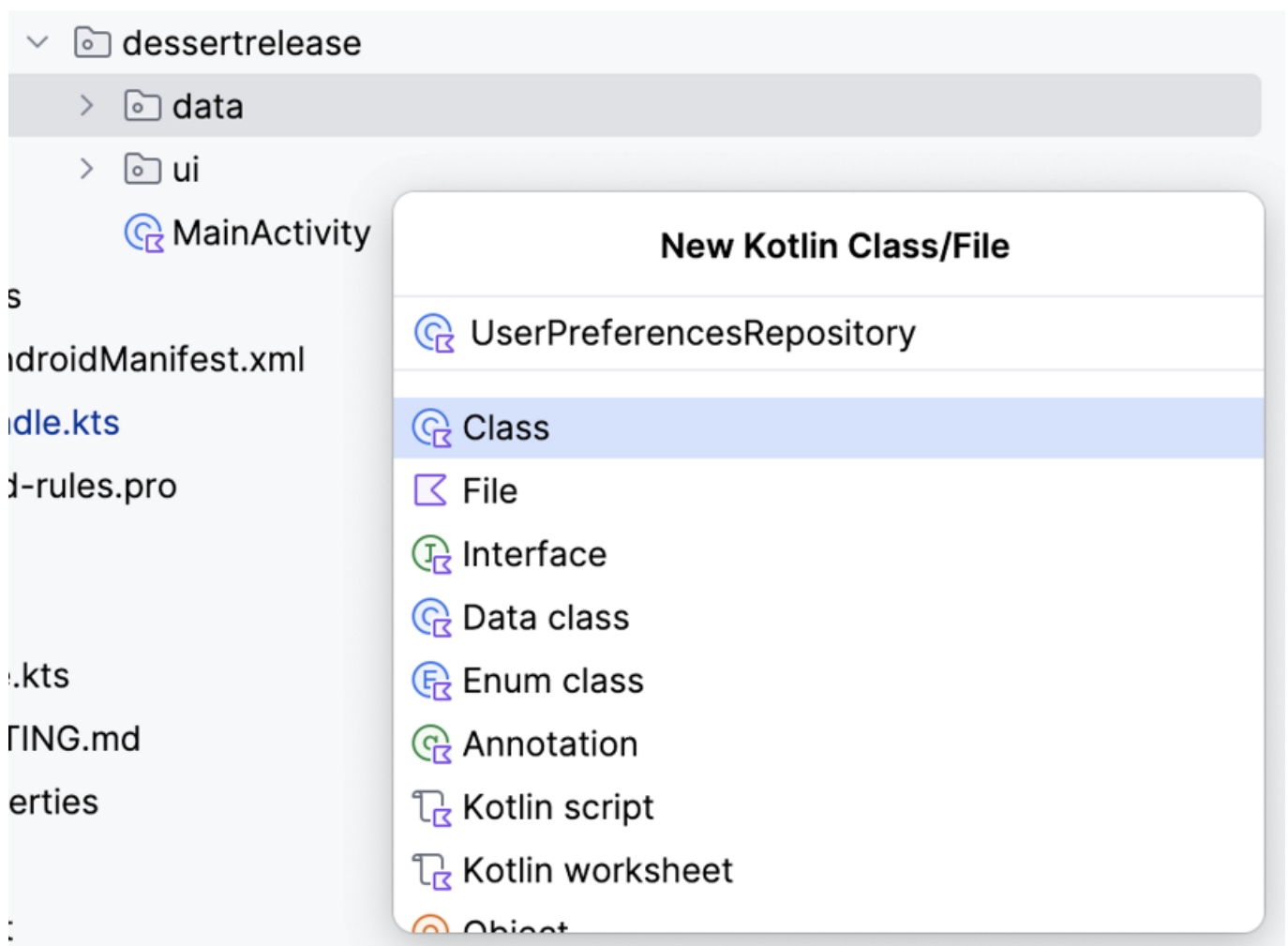
3. Установите зависимости

Добавьте следующее в зависимости в файл app/build.gradle.kts:

```
implementation («androidx.datastore:datastore-preferences:1.0.0»)
```

4. Реализация хранилища пользовательских предпочтений

В пакете данных создайте новый класс UserPreferencesRepository.



В конструкторе UserPreferencesRepository определите приватное свойство value для представления экземпляра объекта DataStore с типом Preferences.

```
class UserPreferencesRepository(
    private val datastore: DataStore<Preferences>
){
}
```

Примечание: Обязательно используйте импорт `androidx.datastore.preferences.core.Preferences` для класса `Preferences`.

`DataStore` хранит пары ключ-значение. Чтобы получить доступ к значению, необходимо определить ключ.

Создайте объект-компаньон внутри класса `UserPreferencesRepository`. Используйте функцию `booleanPreferencesKey()` для определения ключа и передайте ему имя `is_linear_layout`. По аналогии с именами таблиц SQL, ключ должен использовать формат подчеркивания. Этот ключ используется для доступа к булевому значению, указывающему, следует ли показывать линейный макет.

```
class UserPreferencesRepository(
    private val datastore: DataStore<Preferences>
){
    private объект-компаньон {
        val IS_LINEAR_LAYOUT = booleanPreferencesKey(«is_linear_layout»)
    }
    ...
}
```

Запись в хранилище данных Вы создаете и изменяете значения в `DataStore`, передавая лямбду методу `edit()`. Лямбде передается экземпляр `MutablePreferences`, который вы можете использовать для обновления значений в `DataStore`. Все обновления внутри этой лямбды выполняются как одна транзакция. Другими словами, обновление является атомарным - оно происходит в один момент времени. Такой тип обновления предотвращает ситуацию, когда одни значения обновляются, а другие нет.

Создайте функцию приостановки и назовите ее `saveLayoutPreference()`. В функции `saveLayoutPreference()` вызовите метод `edit()` для объекта `dataStore`.

```
``kt suspend fun saveLayoutPreference(isLinearLayout: Boolean) { datastore.edit {
```

```
    }
```

```
}
```

Чтобы сделать код более читабельным, определите имя для свойства `MutablePreferences`, указанного в теле лямбды. Используйте это свойство для установки значения с определенным вами ключом и булевым значением, переданным в

```

функцию saveLayoutPreference().

```kt
suspend fun saveLayoutPreference(isLinearLayout: Boolean) {
 datastore.edit { preferences ->
 preferences[IS_LINEAR_LAYOUT] = isLinearLayout
 }
}

```

Примечание: Значение не существует в DataStore до тех пор, пока не будет вызвана эта функция и не будет установлено значение. Если задать пару ключ-значение в методе edit(), значение будет определено и инициализировано до тех пор, пока не будет очищен кэш приложения или данные.

Чтение из хранилища данных Теперь, когда вы создали способ записи isLinearLayout в хранилище данных, выполните следующие действия для его чтения:

Создайте в UserPreferencesRepository свойство типа Flow под названием isLinearLayout.

```

val isLinearLayout: Flow<Boolean> =

```

Вы можете использовать свойство DataStore.data для отображения значений DataStore. Установите isLinearLayout в свойство data объекта DataStore.

```

val isLinearLayout: Flow<Boolean> = datastore.data

```

Примечание: Этот код не компилируется, а инструкция datastore.data подчеркнута красным цветом. Такой результат ожидаем, поскольку реализация еще не завершена.

Свойство data представляет собой поток объектов Preferences. Объект Preferences содержит все пары ключ-значение в DataStore. Каждый раз, когда данные в DataStore обновляются, в поток выдается новый объект Preferences.

Используйте функцию map для преобразования Flow в Flow. Эта функция принимает лямбду с текущим объектом Preferences в качестве параметра. Вы можете указать ключ, который вы ранее определили для получения предпочтения макета. Имейте в виду, что значение может не существовать, если saveLayoutPreference еще не был вызван, поэтому вы также должны указать значение по умолчанию.

Укажите true, чтобы по умолчанию использовался линейный вид макета.

Примечание: Помните, что пока предпочтение не определено и не инициализировано, оно не существует в DataStore. Поэтому вы должны программно подтвердить, что предпочтение существует, и указать значение по умолчанию, если его нет.

```

val isLinearLayout: Flow<Boolean> = datastore.data.map { preferences ->
 preferences[IS_LINEAR_LAYOUT] ?: true
}

```

```
}
```

Обработка исключений Каждый раз, когда вы взаимодействуете с файловой системой на устройстве, существует вероятность того, что что-то может не сработать. Например, файл может не существовать, диск может быть переполнен или размонтирован. Поскольку DataStore считывает и записывает данные из файлов, при обращении к DataStore могут возникать `IOExceptions`. Для перехвата исключений и обработки этих сбоев используется оператор `catch{}.`

В объекте-компаньоне реализуйте неизменяемое свойство `TAG string`, которое будет использоваться для ведения журнала.

```
private companion object {
 val IS_LINEAR_LAYOUT = booleanPreferencesKey(«is_linear_layout»)
 const val TAG = «UserPreferencesRepo»
}
```

`Preferences DataStore` выбрасывает `IOException` при возникновении ошибки при чтении данных. В блоке инициализации `isLinearLayout`, перед `map()`, используйте оператор `catch{}.` для перехвата `IOException`.

```
val isLinearLayout: Flow<Boolean> = datastore.data
 .catch {}
 .map { preferences ->
 preferences[IS_LINEAR_LAYOUT] ?: true
 }
```

В блоке `catch`, если возникло `IOException`, запишите ошибку в журнал и выдайте `emptyPreferences()`. Если возникло исключение другого типа, предпочтите повторное возникновение этого исключения. Выдавая `emptyPreferences()` в случае ошибки, функция `map` все равно может сопоставить значение по умолчанию.

```
val isLinearLayout: Flow<Boolean> = datastore.data
 .catch {
 if(it is IOException) {
 Log.e(TAG, «Ошибка чтения предпочтений.», it)
 emit(emptyPreferences())
 } else {
 бросить
 }
 }
 .map { preferences ->
 preferences[IS_LINEAR_LAYOUT] ?: true
 }
```

## 5. Инициализация хранилища данных

В этом коделабе вы должны вручную обработать инъекцию зависимостей. Поэтому вы должны вручную предоставить классу `UserPreferencesRepository` хранилище данных `Preferences DataStore`. Выполните следующие шаги, чтобы внедрить `DataStore` в `UserPreferencesRepository`.

Найдите пакет `dessertrelease`. В этом каталоге создайте новый класс `DessertReleaseApplication` и реализуйте класс `Application`. Это контейнер для вашего `DataStore`.

```
class DessertReleaseApplication: Application() {
}
```

Внутри файла `DessertReleaseApplication.kt`, но вне класса `DessertReleaseApplication`, объявите приватную `const val` под названием `LAYOUT_PREFERENCE_NAME`. Присвойте переменной `LAYOUT_PREFERENCE_NAME` строковое значение `layout_preferences`, которое затем можно использовать в качестве имени хранилища данных предпочтений, которое вы инстанцируете на следующем шаге.

```
private const val LAYOUT_PREFERENCE_NAME = «layout_preferences»
```

Все еще вне тела класса `DessertReleaseApplication`, но в файле `DessertReleaseApplication.kt`, создайте частное свойство `value` типа `DataStore` под названием `Context.dataStore`, используя делегат `preferencesDataStore`. Передайте `LAYOUT_PREFERENCE_NAME` в качестве параметра `name` делегата `preferencesDataStore`.

```
private const val LAYOUT_PREFERENCE_NAME = «layout_preferences»
private val Context.dataStore: DataStore<Preferences> by preferencesDataStore(
 name = LAYOUT_PREFERENCE_NAME
)
```

Внутри тела класса `DessertReleaseApplication` создайте экземпляр `lateinit var` хранилища `UserPreferencesRepository`.

```
private const val LAYOUT_PREFERENCE_NAME = «layout_preferences»
private val Context.dataStore: DataStore<Preferences> by preferencesDataStore(
 name = LAYOUT_PREFERENCE_NAME
)

class DessertReleaseApplication: Application() {
 lateinit var userPreferencesRepository: UserPreferencesRepository
}
```

Переопределите метод `onCreate()`.

```
private const val LAYOUT_PREFERENCE_NAME = «layout_preferences»
private val Context.dataStore: DataStore<Preferences> by preferencesDataStore(
 name = LAYOUT_PREFERENCE_NAME
)

class DessertReleaseApplication: Application() {
 lateinit var userPreferencesRepository: UserPreferencesRepository

 override fun onCreate() {
 super.onCreate()
 }
}
```

Внутри метода onCreate() инициализируйте userPreferencesRepository, создав UserPreferencesRepository с dataStore в качестве параметра.

```
private const val LAYOUT_PREFERENCE_NAME = "layout_preferences"
private val Context.dataStore: DataStore<Preferences> by preferencesDataStore(
 name = LAYOUT_PREFERENCE_NAME
)

class DessertReleaseApplication: Application() {
 lateinit var userPreferencesRepository: UserPreferencesRepository

 override fun onCreate() {
 super.onCreate()
 userPreferencesRepository = UserPreferencesRepository(dataStore)
 }
}
```

Добавьте следующую строку внутри тега в файле AndroidManifest.xml.

```
<application
 android:name=".DessertReleaseApplication"
 ...
</application>
```

Этот подход определяет класс DessertReleaseApplication как точку входа в приложение. Цель этого кода - инициализировать зависимости, определенные в классе DessertReleaseApplication, перед запуском MainActivity.

## 6. Используйте репозиторий UserPreferencesRepository

---



Предоставьте хранилище ViewModel Теперь, когда UserPreferencesRepository доступен через инъекцию зависимостей, вы можете использовать его в DessertReleaseViewModel.

В DessertReleaseViewModel создайте свойство UserPreferencesRepository в качестве параметра конструктора.

```
class DessertReleaseViewModel(
 private val userPreferencesRepository: UserPreferencesRepository
) : ViewModel() {
 ...
}
```

В объекте-компаньоне ViewModel в блоке инициализатора viewModelFactory получите экземпляр DessertReleaseApplication, используя следующий код.

```
...
companion object {
 val Factory: ViewModelProvider.Factory = viewModelFactory {
 initializer {
 val application = (this[APPLICATION_KEY] as
DessertReleaseApplication)
 ...
 }
 }
}
```

Создайте экземпляр модели DessertReleaseViewModel и передайте ему хранилище userPreferencesRepository.

```
...
companion object {
 val Factory: ViewModelProvider.Factory = viewModelFactory {
 initializer {
 val application = (this[APPLICATION_KEY] as
DessertReleaseApplication)
 DessertReleaseViewModel(application.userPreferencesRepository)
 }
 }
}
```

Хранилище UserPreferencesRepository теперь доступно для ViewModel. Следующие шаги заключаются в использовании возможностей чтения и записи UserPreferencesRepository, которые вы реализовали ранее.

Сохранение предпочтений макета Отредактируйте функцию `selectLayout()` в `DessertReleaseViewModel`, чтобы получить доступ к хранилищу предпочтений и обновить предпочтение макета. Напомним, что запись в `DataStore` выполняется асинхронно с помощью функции `suspend`. Запустите новую корутину для вызова функции `saveLayoutPreference()` хранилища предпочтений.

```
fun selectLayout(isLinearLayout: Boolean) {
 viewModelScope.launch {
 userPreferencesRepository.saveLayoutPreference(isLinearLayout)
 }
}
```

Чтение предпочтений макета В этом разделе вы рефакторите существующий `uiState: StateFlow` во `ViewModel`, чтобы отразить `isLinearLayout: Flow` из репозитория.

Удалите код, который инициализирует свойство `uiState` в `MutableStateFlow(DessertReleaseUiState)`.

```
val uiState: StateFlow<DessertReleaseUiState> =
```

Предпочтение линейного расположения из хранилища имеет два возможных значения, `true` или `false`, в виде `Flow`. Это значение должно быть сопоставлено с состоянием пользовательского интерфейса.

Установите `StateFlow` в результат преобразования коллекции `map()`, вызванного для потока `isLinearLayout`.

```
val uiState: StateFlow<DessertReleaseUiState> =
 userPreferencesRepository.isLinearLayout.map { isLinearLayout ->
 }
```

Возвращает экземпляр класса данных `DessertReleaseUiState`, передавая булево значение `isLinearLayout`. Экран использует это состояние пользовательского интерфейса для определения правильных строк и иконок для отображения.

```
val uiState: StateFlow<DessertReleaseUiState> =
 userPreferencesRepository.isLinearLayout.map { isLinearLayout ->
 DessertReleaseUiState(isLinearLayout)
 }
```

`UserPreferencesRepository.isLinearLayout` - это `Flow`, который является холодным. Однако для предоставления состояния пользовательскому интерфейсу лучше использовать горячий поток, например `StateFlow`, чтобы состояние всегда было доступно пользовательскому интерфейсу немедленно.

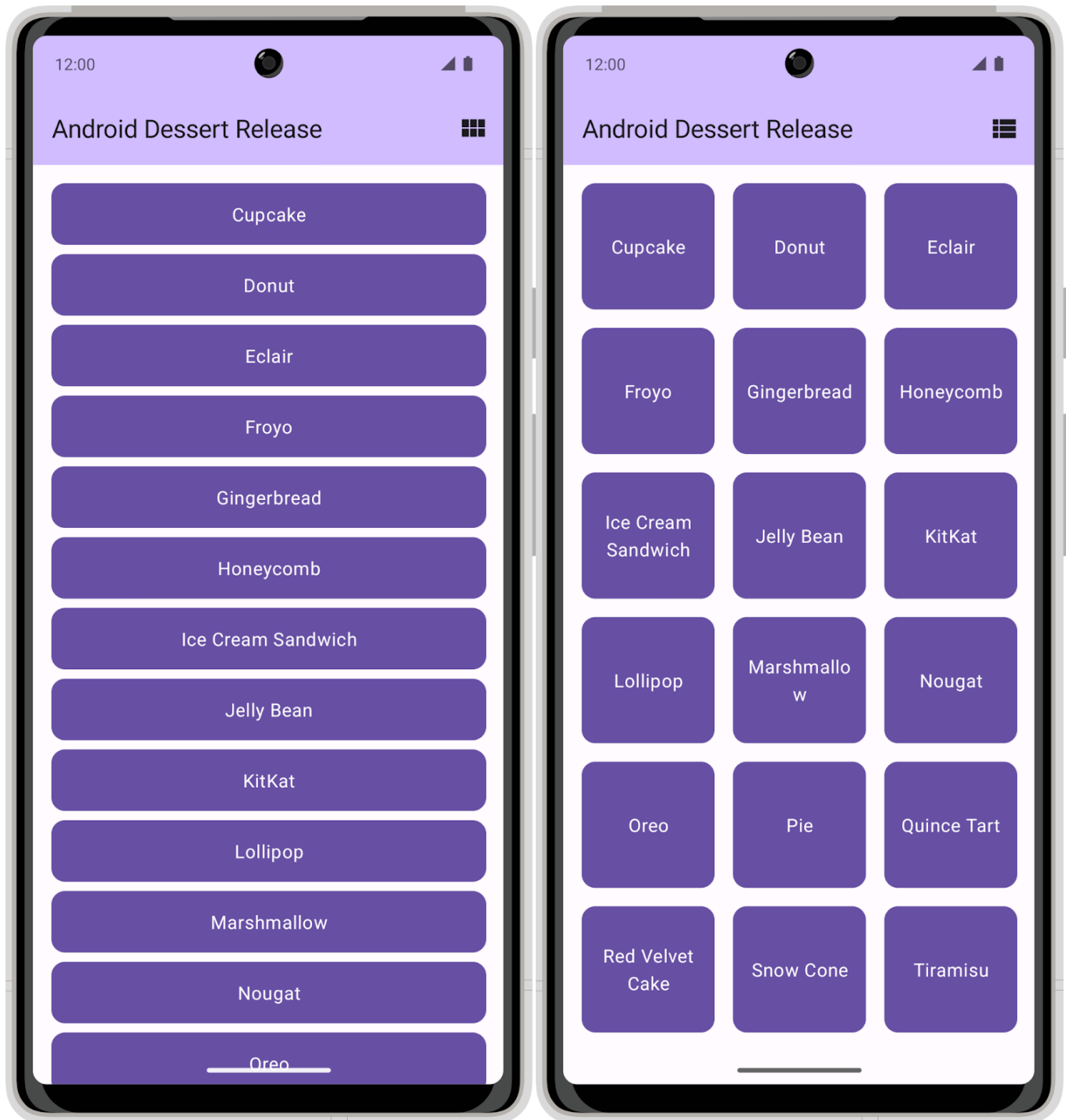
Используйте функцию `stateIn()`, чтобы преобразовать `Flow` в `StateFlow`. Функция `stateIn()` принимает три параметра: `scope`, `started` и `initialValue`. Для этих параметров передайте `viewModelScope`, `SharingStarted.WhileSubscribed(5_000)` и `DessertReleaseUiState()` соответственно.

```
val uiState: StateFlow<DessertReleaseUiState> =
 userPreferencesRepository.isLinearLayout.map { isLinearLayout ->
 DessertReleaseUiState(isLinearLayout)
 }
 .stateIn(
 scope = viewModelScope,
 started = SharingStarted.WhileSubscribed(5_000),
 initialValue = DessertReleaseUiState()
)
```

Примечание: Прочитайте статью [Переход с LiveData на Kotlin's Flow](#), чтобы узнать больше о параметре `started` и о том, почему ему передается `SharingStarted.WhileSubscribed(5_000)`.

Запустите приложение. Обратите внимание, что вы можете нажать на значок переключения, чтобы переключиться между макетом сетки и линейным макетом.

Примечание: Попробуйте переключить макет и закрыть приложение. Снова откройте приложение и заметите, что настройки макета были сохранены.



Поздравляем! Вы успешно добавили Preferences DataStore в свое приложение, чтобы сохранить предпочтения пользователя в макете.

### Получение кода решения

Чтобы загрузить код готового коделаба, вы можете использовать эти git-команды:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-dessert-release.git
$ cd basic-android-kotlin-compose-training-dessert-release
$ git checkout main
```